

Lecture 14. More monads and applicatives

Functional Programming

Utrecht University

Goals

- Introduce the `State` monad
- Use the monad laws to check a proposed implementation
- Map functions of several arguments using applicatives
- Understand difference functor/applicative/monad

Chapter 12.2 from Hutton's book

The State monad

The State monad

```
get :: State s s
put :: s -> State s ()
modify :: (s -> s) -> State s ()
runState :: State s a -> s -> (a, s)
```

`State s a` produces a result of type `a` and a new state of type `s`. The pair is always **(result, state)**.

Reverse Polish Notation (RPN)

Notation in which an operator follows its operands

```
   3 4 + 2 * 10 -
=      7 2 * 10 -
=         14 10 -
=               4
```

Parentheses are not needed when using RPN

A stack machine can evaluate RPN one instruction at a time.

RPN expressions

Expressions in RPN are lists of numbers and operations

```
data Instr = Number Float | Operation ArithOp
type RPN   = [Instr]
```

We reuse the `ArithOp` type from arithmetic expressions

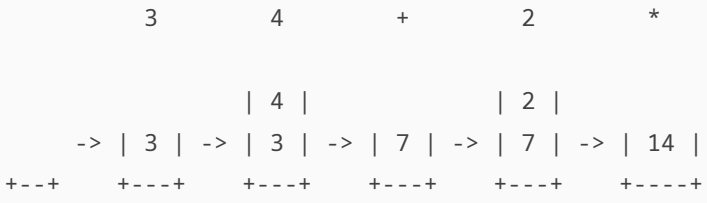
For example, `3 4 + 2 *` becomes

```
[ Number 3, Number 4, Operation Plus
, Number 2, Operation Times ]
```

RPN calculator

To compute the value of an expression in RPN, you keep a stack of values

- Each number is added at the top of the stack
- Operations use the top-most elements in the stack



Case study: RPN calculator

```haskell
type Stack = [Float]

evalInstr :: Instr -> Stack -> Stack
```

The head is the top of the stack. Assume valid input: every operation has at least two operands available.

Case study: RPN calculator

```
type Stack = [Float]

evalInstr :: Instr -> Stack -> Stack
evalInstr (Number f)          stack
      = f : stack
evalInstr (Operation op) (x:y:stack)
      = evalOp op y x : stack
        where evalOp ...
```

The first value popped is the **right** operand: `3 4 -` means `3 - 4`.

Case study: RPN calculator

Let me introduce two new operations to make clear what is going on with the stack

```
pop :: Stack -> (Float, Stack)
push :: Float -> Stack -> Stack
```

Using those the evaluator takes an intuitive form.

Case study: RPN calculator

Let me introduce two new operations to make clear what is going on with the stack

```
pop :: Stack -> (Float, Stack)
pop (x:xs) = (x, xs)
push :: Float -> Stack -> Stack
push x xs  = x : xs
```

Using those the evaluator takes this form:

```
evalInstr (Number f)     s
  = push f s
evalInstr (Operation op) s
  = let (x, s1) = pop s
        (y, s2) = pop s1
    in push (evalOp op y x) s2
```

Encoding state explicitly

A function like pop

```
pop :: Stack -> (Float, Stack)
```

can be seen as a function which modifies a state:

- Takes the original state as an argument
- Returns the new state along with the result

Encoding state explicitly

A function like pop

```
pop :: Stack -> (Float, Stack)
```

can be seen as a function which modifies a state:

- Takes the original state as an argument
- Returns the new state along with the result

A simplified model of `IO`:

```
type IO a = World -> (a, World)
```

Encoding state explicitly

Functions which only update the state return `()`

```
push      :: Float -> Stack -> ((), Stack)
push f s = ((), f : s)

evalInstr :: Instr -> Stack -> ((), Stack)
evalInstr (Number f)     s
  = push f s
evalInstr (Operation op) s
  = let (x, s1) = pop s
        (y, s2) = pop s1
    in push (evalOp op y x) s2
```

Looking for similarities

The same pattern occurs twice in the previous code

```
let (x, newStack) = f oldStack
in  _ -- something which uses x and the newStack
```

This leads to a higher-order function

```
next :: (Stack -> (a, Stack))
     -> (a -> Stack -> (b, Stack))
     -> (Stack -> (b, Stack))
```

Looking for similarities

The same pattern occurs twice in the previous code

```
let (x, newStack) = f oldStack
in  _ -- something which uses x and the newStack
```

This leads to a higher-order function

```
next :: (Stack -> (a, Stack))
     -> (a -> Stack -> (b, Stack))
     -> (Stack -> (b, Stack))
next f g = \s -> let (x, s') = f s
                 in  g x s'
```

(Almost) the State monad

```haskell
type State a = Stack -> (a, Stack)
```

`State` is almost a monad, we only need a `return`

- The type has only one hole, as required

The missing part is a `return` function

- What can we do?

```haskell
return :: a -> Stack -> (a, Stack)
```

(Almost) the State monad

```
type State a = Stack -> (a, Stack)
```

`State` is almost a monad, we only need a `return`

- The type has only one hole, as required

The missing part is a `return` function

- To satisfy the monad laws, `return` must leave the state unchanged

```
return :: a -> Stack -> (a, Stack)
return x = \s -> (x, s)
```

Nicer code for the examples

```
evalInstr :: Instr -> State ()
...
evalInstr (Operation op)
  = do x <- pop
       y <- pop
       push (evalOp op y x)
...
```

The `Stack` value is threaded implicitly

- Similar to a single mutable variable, implemented with pure functions

Notes on implementation

We can generalize this idea to any type `s` of `State`

```
type State s a = s -> (a, s)
```

Notes on implementation

We can generalize this idea to any type `s` of `State`

```
type State s a = s -> (a, s)
```

Alas, if you try to write the instance GHC complains

```
instance Monad (State s) where  -- Wrong!
```

This is because you are only allowed to use a type synonym with *all* arguments applied

- But you need to leave one out to make it a monad

Notes on implementation

The “trick” is to wrap the value in a data type

```
newtype State s a = S (s -> (a, s))

evalState :: State s a -> s -> a
evalState = ???
```

Notes on implementation

The “trick” is to wrap the value in a data type

```haskell
newtype State s a = S (s -> (a, s))

evalState :: State s a -> s -> a
evalState (S f) s = fst (f s)
```

Bind threads the state. Required `Functor` and `Applicative` instances are omitted.

```haskell
instance Monad (State s) where
  return x = S $ \s -> (x, s)
  (S f) >>= g = S $ \s -> let (x, s') = f s
                              S g' = g x
                          in g' s'
```

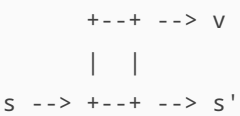
What is going on?

State passing style!

Warning: the following slides contain ASCII-art

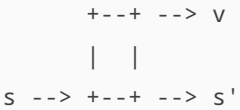
What is going on?

A `State s` a value is a “box” which, once fed a state, gives back a value and the modified state

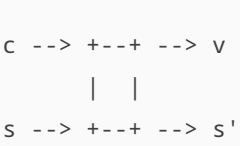


What is going on?

A `State s a` value is a “box” which, once fed a state, gives back a value and the modified state



A function `c -> State s a` is a “box” with an extra input



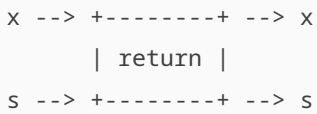
What is going on with `return`?

`return` has type `a -> State s a`

What is going on with `return`?

`return` has type `a -> State s a`

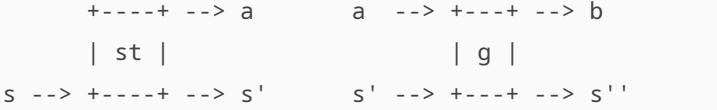
- It is thus a box of the second kind
- It just passes the information through, unmodified



What is going on with `(>>=)`?

```
(>>=) :: State s a -> (a -> State s b) -> State s b
```

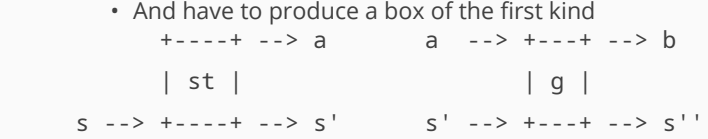
- We take one box of each kind
- And have to produce a box of the first kind



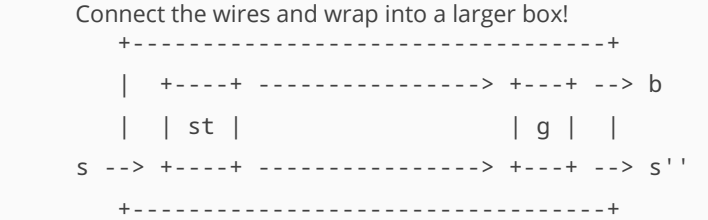
What is going on with `(>>=)`?

```
(>>=) :: State s a -> (a -> State s b) -> State s b
```

- We take one box of each kind
- And have to produce a box of the first kind



Connect the wires and wrap into a larger box!



Another use for state: counters

Given a binary tree, label its nodes in **in-order**: left subtree, node, right subtree

```
> let t = Node (Node Leaf 'a' Leaf)
               'b'
               (Node Leaf 'c' Leaf)
> label t
Node (Node Leaf (0, 'a') Leaf)
     (1, 'b')
     (Node Leaf (2, 'c') Leaf)
```

Another use for state: counters

Given a binary tree, label its nodes in **in-order**: left subtree, node, right subtree

```
> let t = Node (Node Leaf 'a' Leaf)
               'b'
               (Node Leaf 'c' Leaf)
> label t
Node (Node Leaf (0, 'a') Leaf)
     (1, 'b')
     (Node Leaf (2, 'c') Leaf)
```

What is the type for such a function?

Another use for state: counters

Given a binary tree, label its nodes in **in-order**: left subtree, node, right subtree

```
> let t = Node (Node Leaf 'a' Leaf)
               'b'
               (Node Leaf 'c' Leaf)
> label t
Node (Node Leaf (0, 'a') Leaf)
     (1, 'b')
     (Node Leaf (2, 'c') Leaf)
```

What is the type for such a function?

```
label :: Tree a -> Tree (Int, a)
```

Idea: use an implicit counter to keep track of the label

Cooking `label`

The main work happens in a local function which is stateful

```
label' :: Tree a -> State Int (Tree (Int, a))
```

The purpose of `label` is to initialize the state to `0`

```
label t = evalState (label' t) 0
  where label' = ...
```

Cooking `label'`

We use an auxiliary function to get the current label and update it to the next value

```
nextLabel :: State Int Int
nextLabel = S $ \i -> (i, i + 1)
```

Armed with it, writing the stateful `label'` is easy

```
label' Leaf         = return Leaf
label' (Node l x r) = do l' <- label' l
                         i  <- nextLabel
                         r' <- label' r
                         return (Node l' (i, x) r')
```

Monad laws

Valid monads obey these equations between computations:

```
-- Left identity
return x >>= f = f x
```

Monad laws

Valid monads obey these equations between computations:

```
-- Left identity
return x >>= f = f x

-- Right identity
m >>= return = m
```

Monad laws

Valid monads obey these equations between computations:

```
-- Left identity
return x >>= f = f x

-- Right identity
m >>= return = m

-- Associativity: regrouping preserves the order of effects
(m >>= f) >>= g = m >>= (\x -> f x >>= g)
```

Challenge: for the unwrapped stack version, `return x = \_ -> (x, [])` has the right type. Which law rules it out? Give a counterexample.

Summary of monads

Different monads provide different capabilities

- `Maybe` monad models optional values and failure
- `State` monad threads an implicit value
- `[]` monad models search and non-determinism
- `IO` monad provides impure input/output

Summary of monads

Different monads provide different capabilities

- `Maybe` monad models optional values and failure
- `State` monad threads an implicit value
- `[]` monad models search and non-determinism
- `IO` monad provides impure input/output

There are even more monads!

- `Either` models failure, but remembers the problem
- `Reader` provides a read-only environment
- `Writer` computes an on-going value
  - For example, a log of the execution
- `STM` provides atomic transactions
- `Cont` provides non-local control flow

Summary of monads

Monads provide a common interface

- do-notation is applicable to all of them
- Many utility functions (to be described)

Applicatives

Lifting functions

When explaining `Maybe` and `IO` we introduced `liftM2`

```
liftM2 :: (a -> b -> c)
       -> Maybe a -> Maybe b -> Maybe c
liftM2 :: (a -> b -> c)
       -> IO    a -> IO    b -> IO    c
```

In general, we can write `liftM2` for any monad

```
liftM2 :: Monad m => (a   -> b   -> c)
                  ->  m a -> m b -> m c
liftM2 f x y = ???
```

Lifting functions

When explaining `Maybe` and `IO` we introduced `liftM2`

```
liftM2 :: (a -> b -> c)
       -> Maybe a -> Maybe b -> Maybe c
liftM2 :: (a -> b -> c)
       -> IO    a -> IO    b -> IO    c
```

In general, we can write `liftM2` for any monad

```
liftM2 :: Monad m => (a   -> b   -> c)
                  ->  m a -> m b -> m c
liftM2 f x y = do x' <- x
                  y' <- y
                  return (f x' y')
```

Lifting functions

This makes the code shorter and easier to read

```
-- Using do notation
do fn' <- validateFirstName fn
   ln' <- validateLastName  ln
   return (Person fn' ln')

-- Using lift
liftM2 Person (validateFirstName fn)
              (validateLastName  ln)
```

Lifting with different number of arguments

```
liftM :: Monad m => (a -> b) -> m a -> m b
liftM3 :: Monad m => (a -> b -> c -> d)
       -> m a -> m b -> m c -> m d
liftM4 :: ...
```

Lifting with different number of arguments

```
liftM :: Monad m => (a -> b) -> m a -> m b
liftM3 :: Monad m => (a -> b -> c -> d)
       -> m a -> m b -> m c -> m d
liftM4 :: ...
```

The implementation of `liftM` follows the same pattern

```
liftM3 f x y z = do x' <- x
                    y' <- y
                    z' <- z
                    return (f x' y' z')
```

Lifting with different number of arguments

```
liftM :: Monad m => (a -> b) -> m a -> m b
liftM3 :: Monad m => (a -> b -> c -> d)
       -> m a -> m b -> m c -> m d
liftM4 :: ...
```

The implementation of `liftM` follows the same pattern

```
liftM3 f x y z = do x' <- x
                    y' <- y
                    z' <- z
                    return (f x' y' z')
```

Can you find a nicer implementation for `liftM`?

Lifting with different number of arguments

```
liftM :: Monad m => (a -> b) -> m a -> m b
liftM3 :: Monad m => (a -> b -> c -> d)
       -> m a -> m b -> m c -> m d
liftM4 :: ...
```

The implementation of `liftM` follows the same pattern

```
liftM3 f x y z = do x' <- x
                    y' <- y
                    z' <- z
                    return (f x' y' z')
```

Can you find a nicer implementation for `liftM`?

```
liftM = fmap
```

Lifting with different number of arguments

This is clearly suboptimal:

- We need to provide different `liftM` with almost the same implementation
- If we refactor the code by adding or removing parameters to a function, we have to change the `liftM` function we use at the call site

Can we do better?

Introducing (<*>)

Suppose we want to lift a function with two arguments:

```
f :: a -> b -> c     x :: f a     y :: f b
```

What type does `fmap f x` have?

Introducing (<*>)

Suppose we want to lift a function with two arguments:

```
f :: a -> b -> c    x :: f a    y :: f b
```

What type does `fmap f x` have?

```
fmap f :: f a -> f (b -> c)
```

We are able to apply the first argument

```
fmap f x :: f (b -> c)
```

The result is not in the form we want

- The function is now *inside* the functor/monad

Introducing (<*>)

To apply the next argument we need some magical function

```
(<*>) :: f (b -> c) -> f b -> f c
```

If we had that function, then we can write

```
fmap f x <*> y
= -- using the synonym (<$>) = fmap
f <$> x <*> y
```

Introducing (<*>)

```
(<*>) :: f (b -> c) -> f b -> f c
```

Note that in the type of `(<*>)` we can choose `c` to be yet another function type

- As a result, by means of `fmap` and `(<*>)` we can lift a function with any number of arguments

```
f  :: a -> b -> ... -> y -> z
ma :: m a
mb :: m b
...
f <$> ma <*> mb <*> ... <*> my :: m z
```

Using (<*>)

Take the `label'` functions for trees we wrote previously

```
label' Leaf         = return Leaf
label' (Node l x r) = do l' <- label' l
                         i  <- nextLabel
                         r' <- label' r
                         return (Node l' (i, x) r')
```

`(<*>)` still threads the state from left to right:

```
label' Leaf = return Leaf
label' (Node l x r)
           = Node <$> label' l
                  <*> ((\i -> (i, x)) <$> nextLabel)
                  <*> label' r
```

Applicatives

It turns out that `(<*>)` by itself is an useful abstraction

- `Functor` allows you to lift one-argument function
- With `(<*>)` you can lift functions with more than one argument

Applicatives

It turns out that `(<*>)` by itself is an useful abstraction

- `Functor` allows you to lift one-argument function
- With `(<*>)` you can lift functions with more than one argument

For completeness, we also want a way to lift 0-ary functions. What is the type of an `fmap` for 0-ary functions?

Applicatives

It turns out that `(<*>)` by itself is an useful abstraction

- `Functor` allows you to lift one-argument function
- With `(<*>)` you can lift functions with more than one argument

For completeness, we also want a way to lift 0-ary functions. What is the type of an `fmap` for 0-ary functions?

A type constructor with these operations is called an **applicative** (functor)

```
class Functor f => Applicative f where
  pure  :: a -> f a
  (<*>) :: f (a -> b) -> f a -> f b
```

Monads are applicatives

Every monad is also an applicative

```
pure = ???
mf <*> mx = ???
```

Monads are applicatives

Every monad is also an applicative

```
pure = return
mf <*> mx = do f <- mf
               x <- mx
               return (f x)
```

Monads are applicatives

Every monad is also an applicative

```
pure = return
mf <*> mx = do f <- mf
               x <- mx
               return (f x)
```

As a result, you can use applicative style with `IO`, `[]`, `State`...

```
do x <- xs          ==  [ x + y
   y <- ys                | x <- xs
   return (x + y)         , y <- ys ]


                    == (+) <$> xs <*> ys
```

Monads are applicatives

Every monad is also an applicative

```
pure = return
mf <*> mx = do f <- mf
               x <- mx
               return (f x)
```

As a result, you can use applicative style with `IO`, `[]`, `State`...

```
do x <- xs          ==  [ x + y
   y <- ys                | x <- xs
   return (x + y)         , y <- ys ]

                   == (+) <$> xs <*> ys
```

But there are applicatives which are not monads!

The functor - applicative - monad hierarchy

```haskell
class Functor f where
  fmap :: (a -> b) -> f a -> f b

class Functor f => Applicative f where
  pure  :: a -> f a
  (<*>) :: f (a -> b) -> f a -> f b

class Applicative f => Monad f where
  -- return is the same as Applicative's pure
  (>>=) :: f a -> (a -> f b) -> f b
```

The functor - applicative - monad hierarchy

```
fmap        :: (a -> b)   -> f a -> f b
(<*>)       :: f (a -> b) -> f a -> f b
flip (>>=) :: (a -> f b) -> f a -> f b
```

- Have seen: can express <*> in terms of >>= and `return`
- Exercise: express `fmap` in terms of <*> and `pure`

The functor - applicative - monad hierarchy

```
fmap        :: (a -> b)   -> f a -> f b
(<*>)       :: f (a -> b) -> f a -> f b
flip (>>=) :: (a -> f b) -> f a -> f b
```

- Have seen: can express <*> in terms of >>= and `return`
- Exercise: express `fmap` in terms of <*> and `pure`

- Bind can use a returned value to choose the next computation.

When do we need bind?

```haskell
-- A
do x <- getLine
   y <- getLine
   return (x, y)

-- B
do x <- getLine
   if null x then return "" else getLine
```

Rewrite A using <$> and <*>. Why does reading twice and then selecting a result fail to preserve B's behaviour?

Summary

- State monad models computation which can read/write some bit of state
- Applicatives are functors + more structure (to lift multiple argument functions)
- Monads are applicatives + more structure (to use an earlier result to choose the next computation)